

Software Design Document (SDD)

Module: TCS3472 Driver

Version: 1.1

Authors: Steven McClellan, Vania Leal, Leonardo García, Melanie Picen

Module files: tcs3472_driver.c / tcs3472_driver.h

1. Introduction

1.1 Purpose

This document describes the design and architecture of the TCS3472 driver module. It provides details on data structures, register access strategy, control flow, helper routines, and public API functions used to initialize, configure, read, and power down a TCS3472 RGB + Clear light-to-digital sensor over I2C.

1.2 Scope

The TCS3472 driver is a low-level embedded software component. It abstracts the sensor command protocol and register map behind a small C API. Higher-level modules can configure gain and integration time, read RGBC samples, read only the Clear channel for exposure control, and power the sensor off without manually forming TCS3472 command bytes or handling register burst reads.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
TCS3472	RGB + Clear color light-to-digital converter accessed through I2C.
RGBC	Four measurement channels: Red, Green, Blue, and Clear.
I2C	Inter-Integrated Circuit serial bus used to communicate with the sensor.
PON	Power ON bit in the ENABLE register.
AEN	RGBC ADC enable bit in the ENABLE register.
ATIME	Integration time register. Lower register values produce longer integration time.
AGAIN	Analog gain setting configured through the CONTROL register.
AVALID	STATUS bit indicating that RGBC data is valid.
Burst read	Multi-byte read using the TCS3472 auto-increment command protocol.
SDD	Software Design Document.

1.4 Requirements

Req. ID	Function	Requirement Statement
FR-1	tcs3472_init()	The system shall verify the sensor ID, power on the oscillator, apply the requested configuration, and enable RGBC conversion.
FR-2	tcs3472_config()	The system shall configure gain and integration time at run time.
FR-3	tcs3472_read_rgbc()	The system shall wait until data is valid and read Clear, Red, Green, and Blue channels in one burst transaction.
FR-4	tcs3472_read_clear()	The system shall wait until data is valid and read only the Clear channel for lightweight control loops.
FR-5	tcs3472_power_off()	The system shall clear the sensor ENABLE register to stop RGBC conversion and power down the device.

NFR-1	All APIs	The driver shall use no dynamic memory allocation and shall return explicit status codes.
NFR-2	All APIs	The driver shall validate required pointers before accessing them.
NFR-3	Internal helpers	The driver shall communicate with the sensor only through the I2C driver functions.

2. System Architecture

2.1 High-Level Design

The driver is located between application/control modules and the I2C hardware abstraction. It owns the TCS3472 register protocol, while the I2C driver owns the physical bus transaction implementation.

Layer	Files	Description
Application / Control Layer	main.c, tcs3472_control.c/.h	Calls the TCS3472 driver to initialize the sensor, adjust gain/integration time, and acquire color data.
TCS3472 Driver Layer	tcs3472_driver.c/.h	Implements sensor-specific register writes, register reads, ID validation, data-ready polling, RGBC reconstruction, and power-off control.
I2C Driver Layer	i2c_driver.c/.h	Provides i2c_writeRegDevice() and i2c_readRegDevice() used by the TCS3472 driver.
Hardware Layer	TCS3472 sensor	Contains ENABLE, ATIME, CONTROL, ID, STATUS, and RGBC data registers.

2.2 Module Interfaces

Public API: Functions declared in tcs3472_driver.h. Dependencies: stdint.h, stddef.h, and i2c_driver.h. The driver expects an initialized I2C_RegDef_t pointer for the target I2C peripheral.

2.3 Class Diagram

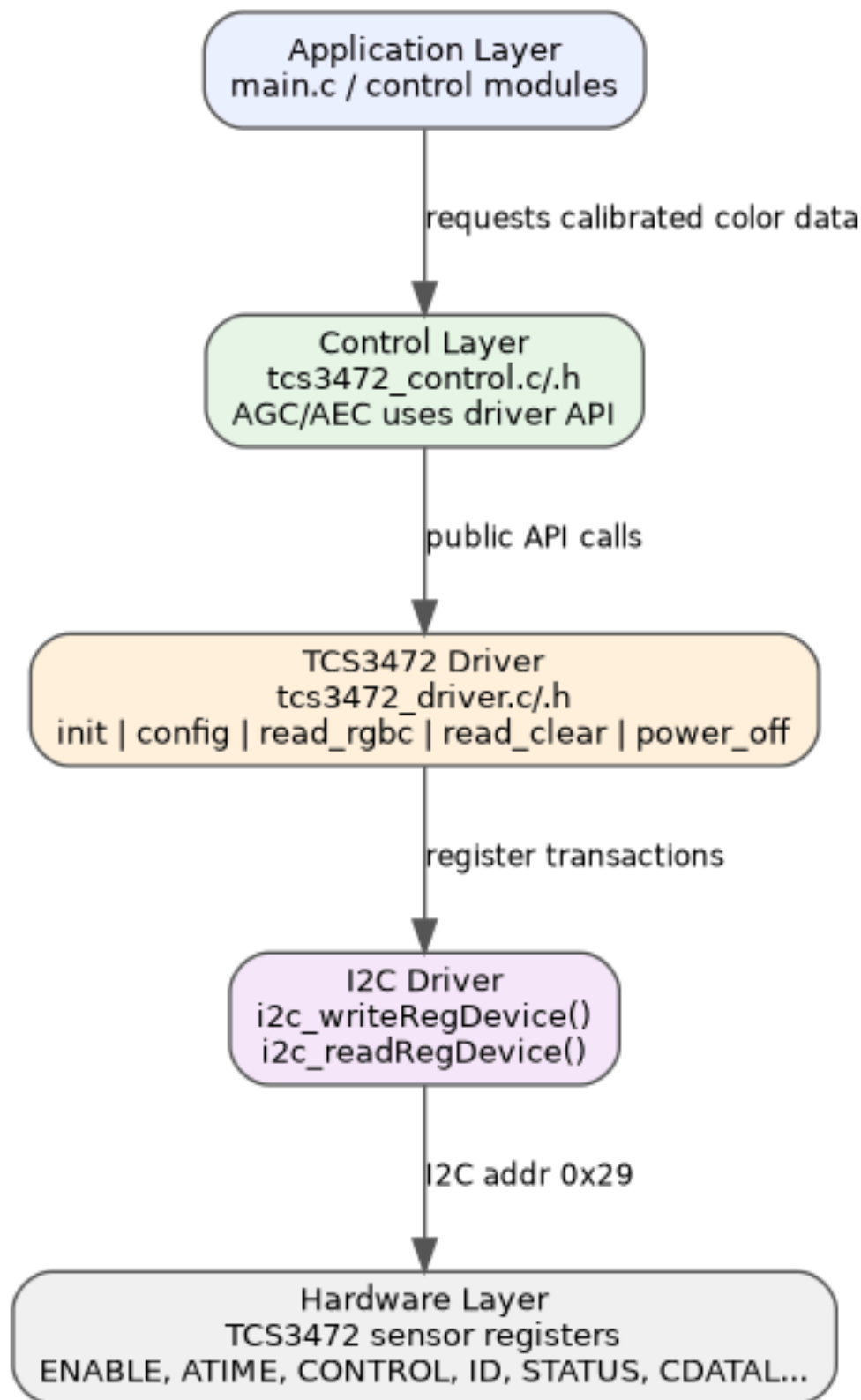


Figure 1. Class Diagram - TCS3472 Driver

2.4 Driver State Machine

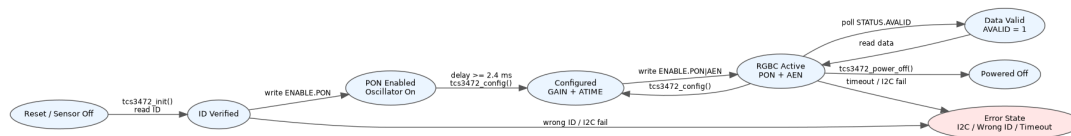


Figure 2. State Diagram - TCS3472 Driver Lifecycle

3. Detailed Design

3.1 Module Behavior

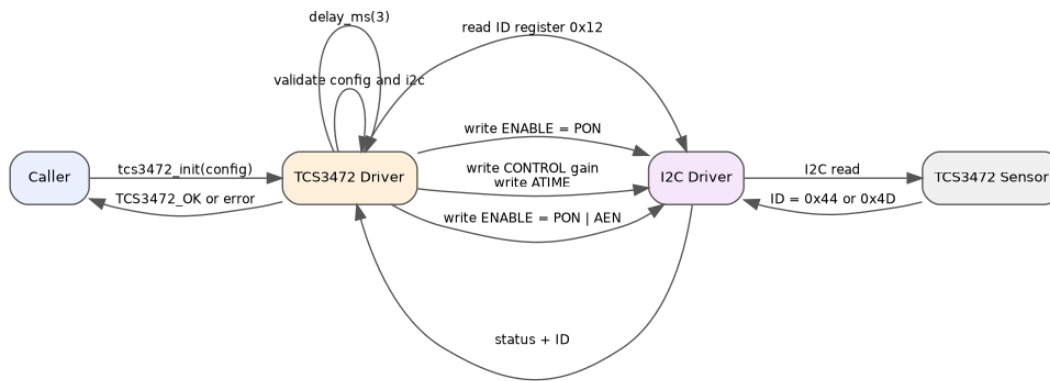


Figure 3. Sequence Diagram - tcs3472_init

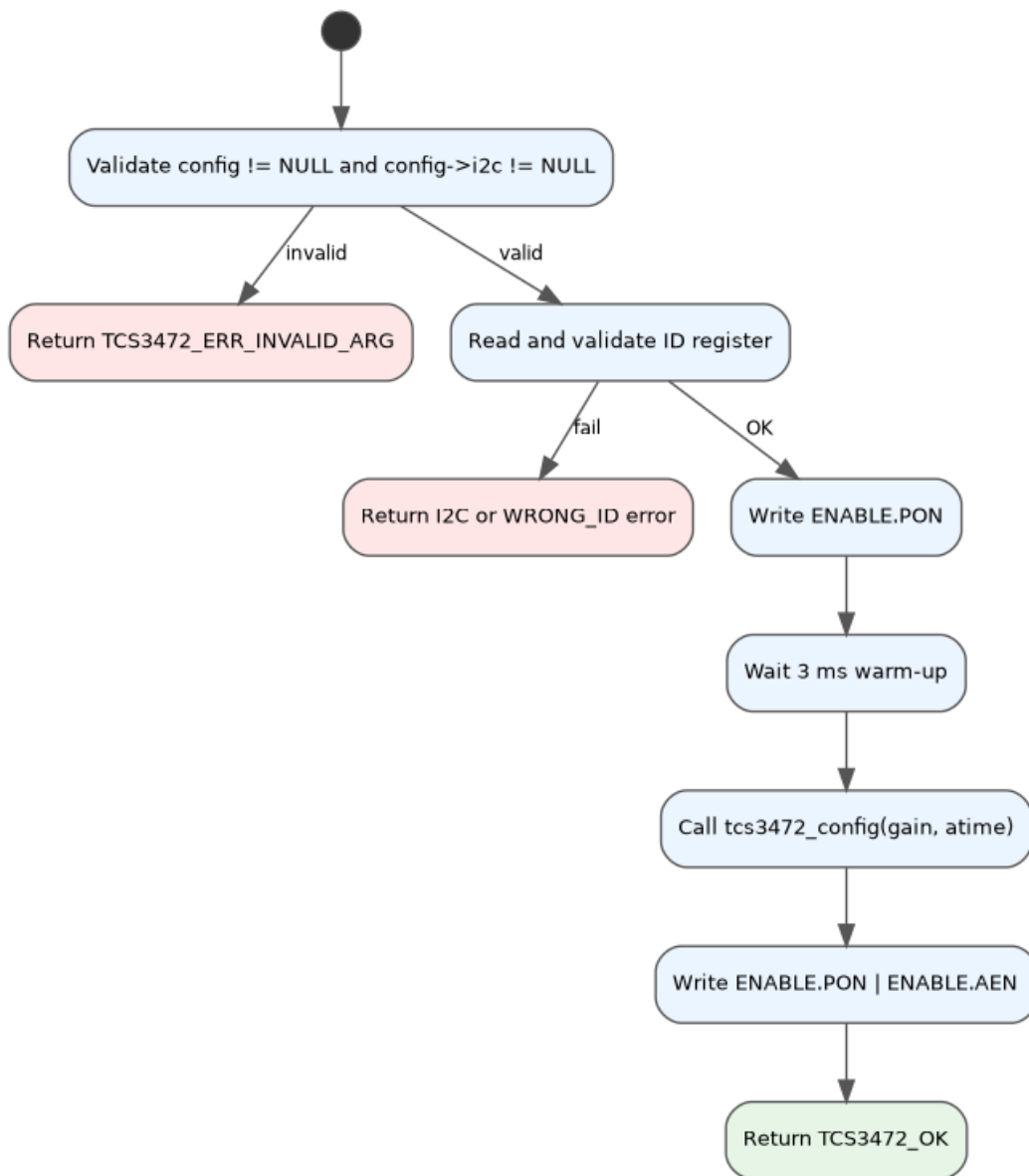


Figure 4. Activity Diagram - tcs3472_init

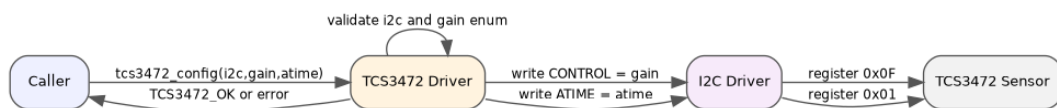


Figure 5. Sequence Diagram - tcs3472_config

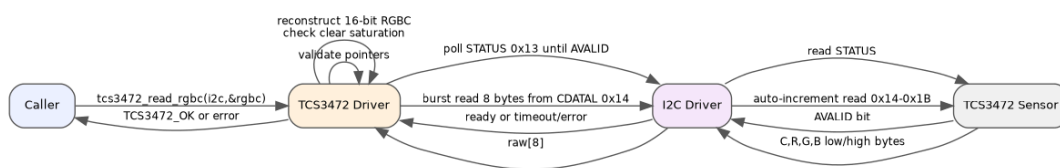


Figure 6. Sequence Diagram - tcs3472_read_rgbc

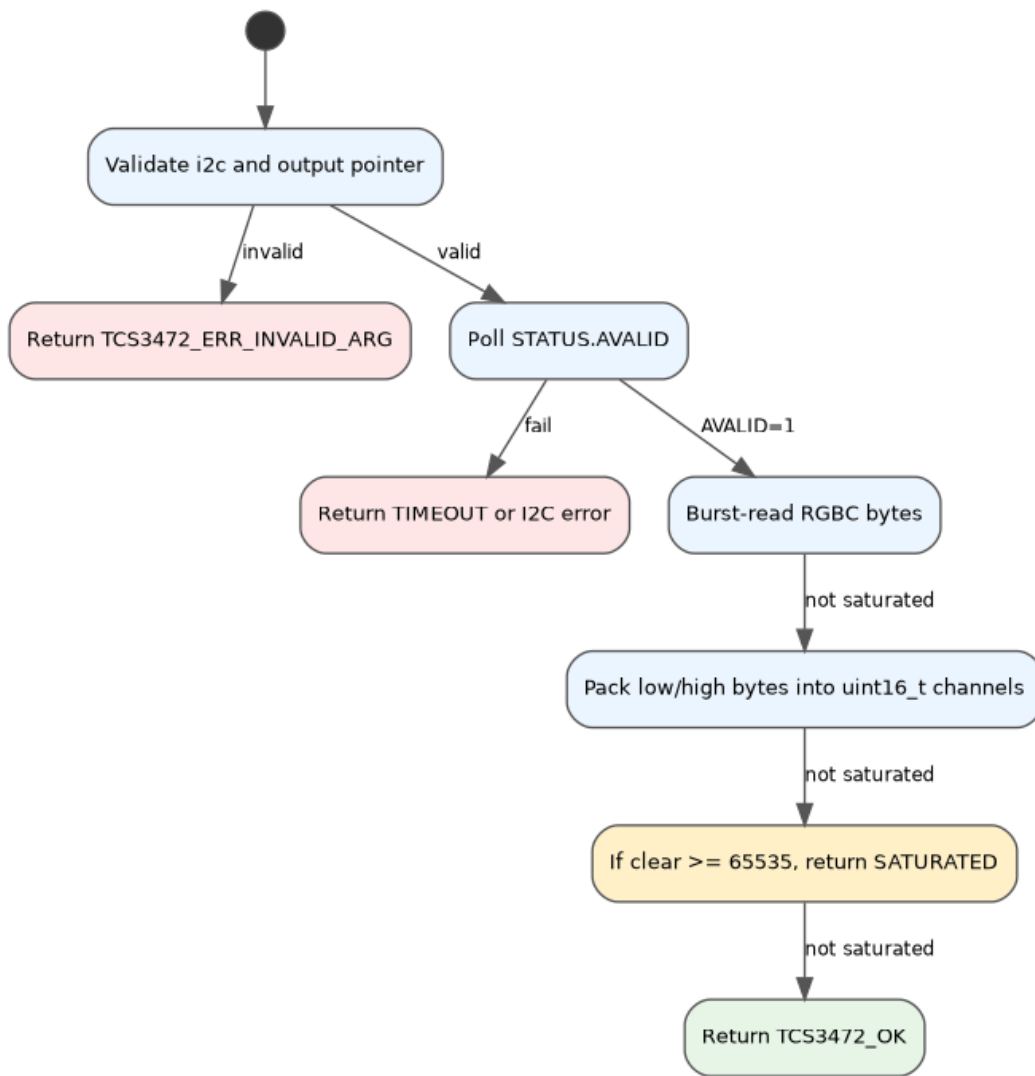


Figure 7. Activity Diagram - `tcs3472_read_rgbc`

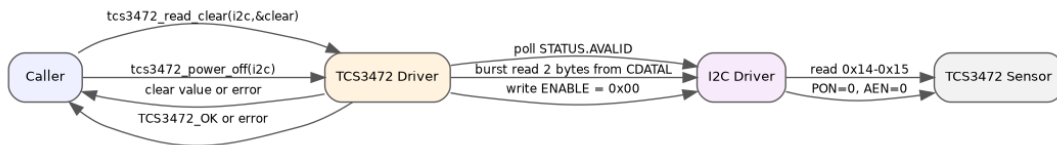


Figure 8. Sequence Diagram - `tcs3472_read_clear` and `tcs3472_power_off`

3.2 TCS3472 Driver

The driver initializes the TCS3472 sensor by validating the configuration pointer, checking the device ID register, enabling the internal oscillator, applying the requested gain and integration time, and enabling the RGBC ADC. The initialization sequence is intentionally conservative: the driver waits approximately 3 ms after setting PON before enabling RGBC conversion, matching the sensor warm-up requirement described in the code comments.

Configuration is done through one function, `tcs3472_config()`, which writes the CONTROL register with the selected gain and the ATIME register with the selected integration time. The function validates that the I2C pointer is non-null and that the gain value matches one of the supported discrete enum values: 1x, 4x, 16x, or 60x.

Data acquisition uses a polling model. Before reading data, the driver repeatedly reads the STATUS register until AVALID is set or a timeout expires. `tcs3472_read_rgbc()` reads 8 consecutive bytes starting at CDATAL and reconstructs four 16-bit values using little-endian byte order. `tcs3472_read_clear()` performs the same readiness check but reads only the 2 Clear-channel bytes for lower overhead.

The driver is blocking and synchronous. It returns after the I2C transactions complete, after the data-ready polling succeeds, or after an error is detected. It does not use interrupts, DMA, an RTOS, or dynamic memory.

4. Application Programming Interfaces

4.1 Data Types and Macros

Macro / Type	Value / Members	Notes
TCS3472_ADDR	0x29U	7-bit I2C address used for TCS34725/27 devices.
TCS3472_CMD(reg)	0xA0U reg	Builds an auto-increment command for multi-byte register reads.
TCS3472_CMD_BYTE(reg)	0x80U reg	Builds a single-byte command for individual register access.
TCS3472_REG_ENABLE	0x00U	Power and RGBC enable register.
TCS3472_REG_ATIME	0x01U	Integration time register.
TCS3472_REG_CONTROL	0x0FU	Gain control register.
TCS3472_REG_ID	0x12U	Device identification register.
TCS3472_REG_STATUS	0x13U	Status register containing AVALID.
TCS3472_REG_CDATAL	0x14U	Start register for Clear low byte and RGBC burst reads.
TCS3472_ENABLE_PON	1U << 0	Power ON bit.
TCS3472_ENABLE_AEN	1U << 1	RGBC ADC enable bit.
TCS3472_STATUS_AVALID	1U << 0	Data-valid flag.
TCS3472_ADC_MAX	65535U	Clear-channel saturation threshold.
tcs3472_gain_t	1x, 4x, 16x, 60x	Discrete analog gain selections.
tcs3472_atime_t	uint8_t	Integration time register type.
tcs3472_rgbc_t	clear, red, green, blue	Stores one complete RGBC measurement as four uint16_t values.
tcs3472_config_t	i2c, gain, atime	Run-time configuration passed to <code>tcs3472_init()</code> .

4.2 Error Codes

Code	Value	Notes / Definition
TCS3472_OK	0	Operation completed successfully.
TCS3472_ERR_INVALID_ARG	-1	A required pointer is NULL or a configuration argument is invalid.
TCS3472_ERR_I2C	-2	The underlying I2C transaction failed.
TCS3472_ERR_WRONG_ID	-3	The ID register did not match an accepted TCS3472 device ID.
TCS3472_ERR_NOT_READY	-4	Defined in the header but not currently returned by the implementation.

TCS3472_ERR_SATURATED	-5	The Clear channel reached the ADC full-scale threshold.
TCS3472_ERR_TIMEOUT	-6	AVALID polling expired before valid data became available.

4.3 Function Definitions

tcs3472_init

Field	Description
Function Name	<code>tcs3472_init</code>
Syntax	<code>tcs3472_status_t tcs3472_init(const tcs3472_config_t *config)</code>
Parameters (in)	config: pointer to a populated <code>tcs3472_config_t</code> containing I2C instance, gain, and ATIME.
Parameters (inout)	None.
Parameters (out)	None.
Return value	<code>tcs3472_status_t</code> : TCS3472_OK, TCS3472_ERR_INVALID_ARG, TCS3472_ERR_WRONG_ID, or TCS3472_ERR_I2C.
Description	Initializes the sensor by checking its ID, enabling PON, waiting for warm-up, applying gain/ATIME, and enabling RGBC conversion.
Constraints	Requires the I2C peripheral to be initialized before this call. The function blocks during the warm-up delay and I2C transactions.
Available via	<code>tcs3472_driver.h</code>

tcs3472_config

Field	Description
Function Name	<code>tcs3472_config</code>
Syntax	<code>tcs3472_status_t tcs3472_config(I2C_RegDef_t *i2c, tcs3472_gain_t gain, tcs3472_atime_t atime)</code>
Parameters (in)	i2c: initialized I2C instance. gain: selected analog gain. atime: integration time register value.
Parameters (inout)	None.
Parameters (out)	None.
Return value	<code>tcs3472_status_t</code> : TCS3472_OK, TCS3472_ERR_INVALID_ARG, or TCS3472_ERR_I2C.
Description	Writes CONTROL and ATIME registers to reconfigure the sensor at run time.
Constraints	Validates gain against the four supported enum values. Any <code>uint8_t</code> ATIME value is accepted.
Available via	<code>tcs3472_driver.h</code>

tcs3472_read_rgbc

Field	Description
Function Name	<code>tcs3472_read_rgbc</code>
Syntax	<code>tcs3472_status_t tcs3472_read_rgbc(I2C_RegDef_t *i2c, tcs3472_rgbc_t *rgbc)</code>
Parameters (in)	i2c: initialized I2C instance.
Parameters (inout)	None.
Parameters (out)	rgbc: output structure filled with Clear, Red, Green, and Blue channel counts.
Return value	<code>tcs3472_status_t</code> : TCS3472_OK, TCS3472_ERR_INVALID_ARG, TCS3472_ERR_TIMEOUT, TCS3472_ERR_I2C, or TCS3472_ERR_SATURATED.
Description	Waits for AVALID, burst-reads 8 bytes from CDATAL through BDATH, and reconstructs four 16-bit channel values.
Constraints	Blocking polling is used. Saturation is checked only against the Clear channel.
Available via	<code>tcs3472_driver.h</code>

tcs3472_read_clear

Field	Description
Function Name	<code>tcs3472_read_clear</code>
Syntax	<code>tcs3472_status_t tcs3472_read_clear(I2C_RegDef_t *i2c, uint16_t *clear)</code>
Parameters (in)	<code>i2c</code> : initialized I2C instance.
Parameters (inout)	None.
Parameters (out)	<code>clear</code> : output pointer filled with the Clear channel value.
Return value	<code>tcs3472_status_t</code> : TCS3472_OK, TCS3472_ERR_INVALID_ARG, TCS3472_ERR_TIMEOUT, or TCS3472_ERR_I2C.
Description	Waits for AVALID and reads only the Clear-channel low/high bytes.
Constraints	Does not perform the saturation check used by <code>tcs3472_read_rgbc()</code> .
Available via	<code>tcs3472_driver.h</code>

tcs3472_power_off

Field	Description
Function Name	<code>tcs3472_power_off</code>
Syntax	<code>tcs3472_status_t tcs3472_power_off(I2C_RegDef_t *i2c)</code>
Parameters (in)	<code>i2c</code> : initialized I2C instance.
Parameters (inout)	None.
Parameters (out)	None.
Return value	<code>tcs3472_status_t</code> : TCS3472_OK, TCS3472_ERR_INVALID_ARG, or TCS3472_ERR_I2C.
Description	Powers down the sensor by writing 0x00 to the ENABLE register.
Constraints	The header comments list OK and I2C error, but the implementation also returns INVALID_ARG when <code>i2c</code> is NULL.
Available via	<code>tcs3472_driver.h</code>

4.4 Internal Function Definitions

delay_ms

Field	Description
Function Name	<code>delay_ms</code>
Syntax	<code>static void delay_ms(uint32_t ms)</code>
Parameters	<code>ms</code> : approximate milliseconds to delay.
Return value	void.
Description	Busy-wait delay using a NOP loop.
Constraints	Timing depends on CPU clock and compiler behavior. The comment assumes about 16 MHz and about 4000 loop iterations per ms.

tcs3472_write_reg

Field	Description
Function Name	<code>tcs3472_write_reg</code>
Syntax	<code>static tcs3472_status_t tcs3472_write_reg(I2C_RegDef_t *i2c, uint8_t reg, uint8_t val)</code>
Parameters	<code>i2c</code> : I2C instance. <code>reg</code> : target register. <code>val</code> : byte to write.
Return value	TCS3472_OK or TCS3472_ERR_I2C.
Description	Writes one byte to a register using TCS3472_CMD_BYTE(<code>reg</code>).
Constraints	Assumes pointer validation has already been handled by caller.

tcs3472_read_reg

Field	Description
Function Name	<code>tcs3472_read_reg</code>
Syntax	<code>static tcs3472_status_t tcs3472_read_reg(I2C_RegDef_t *i2c, uint8_t reg, uint8_t *out)</code>
Parameters	<code>i2c</code> : I2C instance. <code>reg</code> : target register. <code>out</code> : destination byte pointer.
Return value	<code>TCS3472_OK</code> or <code>TCS3472_ERR_I2C</code> .
Description	Reads one byte from a register using <code>TCS3472_CMD_BYTE(reg)</code> .
Constraints	Assumes pointer validation has already been handled by caller.

tcs3472_read_burst

Field	Description
Function Name	<code>tcs3472_read_burst</code>
Syntax	<code>static tcs3472_status_t tcs3472_read_burst(I2C_RegDef_t *i2c, uint8_t reg, uint8_t *buf, size_t len)</code>
Parameters	<code>i2c</code> : I2C instance. <code>reg</code> : starting register. <code>buf</code> : receive buffer. <code>len</code> : byte count.
Return value	<code>TCS3472_OK</code> or <code>TCS3472_ERR_I2C</code> .
Description	Reads consecutive bytes using <code>TCS3472_CMD(reg)</code> auto-increment protocol.
Constraints	Caller must provide a buffer large enough for <code>len</code> bytes.

tcs3472_check_id

Field	Description
Function Name	<code>tcs3472_check_id</code>
Syntax	<code>static tcs3472_status_t tcs3472_check_id(I2C_RegDef_t *i2c)</code>
Parameters	<code>i2c</code> : I2C instance.
Return value	<code>TCS3472_OK</code> , <code>TCS3472_ERR_I2C</code> , or <code>TCS3472_ERR_WRONG_ID</code> .
Description	Reads the ID register and accepts 0x44 or 0x4D.
Constraints	Only verifies the device family ID; it does not distinguish all package variants.

tcs3472_wait_valid

Field	Description
Function Name	<code>tcs3472_wait_valid</code>
Syntax	<code>static tcs3472_status_t tcs3472_wait_valid(I2C_RegDef_t *i2c)</code>
Parameters	<code>i2c</code> : I2C instance.
Return value	<code>TCS3472_OK</code> , <code>TCS3472_ERR_I2C</code> , or <code>TCS3472_ERR_TIMEOUT</code> .
Description	Polls STATUS until AVALID is set or a fixed timeout counter expires.
Constraints	Timeout is loop-count based rather than real-time based.

5. Future Enhancements

- Add a millisecond timer-based timeout instead of a raw loop counter in `tcs3472_wait_valid()`.
- Add optional interrupt-driven data-ready handling if the sensor INT pin is connected.
- Add saturation checking to `tcs3472_read_clear()` for consistency with `tcs3472_read_rgbc()`.
- Document or remove unused status code `TCS3472_ERR_NOT_READY` to keep the API tighter.
- Add unit tests using a mocked I2C driver to verify ID checking, command byte formation, burst reconstruction, and error propagation.

- Add explicit metadata for accepted device variants and a helper to convert ATIME register values to integration time in milliseconds.

6. Implementation Notes

- The driver uses little-endian reconstruction: low byte | (high byte << 8). This matches the TCS3472 data register layout used in the implementation.
- The implementation returns TCS3472_ERR_INVALID_ARG from tcs3472_power_off() when i2c is NULL. The header comment should be updated to mention this return path.
- TCS3472_REG_RDATAL, TCS3472_REG_GDATAL, and TCS3472_REG_BDATAL are defined for clarity even though tcs3472_read_rgbc() starts from CDATAL and reads the full 8-byte block.
- The driver is intentionally lightweight: no heap allocation, no HAL dependency inside this module, and no direct exposure of register transactions to higher layers.